



Actividad 2

Excepciones, Iterables, Iteradores y Lista Ligada

Entrega

- **Lugar:** Repositorio personal de GitHub — Carpeta: Actividades/AC2
- **Fecha máxima de entrega:** 3 de septiembre a las 17:10 hrs
- **Ejecución de actividad:** La Actividad será ejecutada **únicamente** desde la terminal del computador. Los *paths* relativos utilizados en la Actividad deben ser coherentes con esta instrucción, y no pueden modificarse.

Importante: Antes de comenzar, comprueba que Git este funcionando correctamente en tu repositorio privado. Para esto, **sube los archivos base de la actividad de inmediato** (*add*, *commit*, *push*). Se espera que en esta actividad (así como en las demás actividades y tareas) utilices Git a lo largo de **todo tu desarrollo** como una herramienta, no sólo como un método de entrega. Es por esto que recomendamos enfáticamente que vayas subiendo tus cambios constantemente (*push*), ya que **problemas de último minuto** relacionados con la entrega y Git **no serán considerados**.

Introducción

En **DCCFactory**, las cajas avanzan por una cinta transportadora hasta llegar al puesto de inspección. Cada nueva caja se incorpora al final de la cinta y las cajas se procesan respetando su orden de llegada.

Al procesar una caja, esta sale de la cinta y pasa por la máquina de inspección. Si algún dato de la caja no cumple con lo exigido, la máquina la rechaza; si todo está en orden, la aprueba. En ambos casos el resultado queda anotado en la bitácora.

Tu tarea será implementar la cinta transportadora, la máquina encargada de revisar las cajas y las consultas que resumen la bitácora.

Archivos y entidades

```
AC2/
├── clases/
│   ├── fabrica.py
│   ├── lista_ligada.py
│   └── maquinas.py
├── datos/
│   └── cajas.csv
├── consultas.py
├── utils.py
├── main.py
└── salida_esperada.txt
```

■ **Entregar** **Modificar** `clases/lista_ligada.py`

Contiene las clases `Nodo`, `ListaLigada` e `IteradorListaLigada`.

La clase `Nodo` representa un eslabón de la cadena y guarda un elemento. **No debes modificarla**. Posee los siguientes atributos:

- `self.valor` Elemento almacenado en el nodo: una `Caja` o un `Registro`.
- `self.siguiente` Referencia al nodo siguiente. Parte con el valor `None`, ya que el nodo aún no tiene sucesor.

La clase `ListaLigada` es la estructura que encadena esos nodos. Se usa dos veces: como cinta y como bitácora. Posee los siguientes atributos:

- `self.cabeza` Primer nodo de la lista. Parte con el valor `None`.
- `self cola` Último nodo de la lista. Parte con el valor `None`.
- `self.largo` Cantidad de elementos almacenados. Parte en cero.

La clase `IteradorListaLigada` se encarga de recorrer la `ListaLigada` nodo por nodo. Sus atributos los defines tú: necesita recordar en qué punto va el recorrido.

■ **Entregar** **Modificar** `clases/maquinas.py`

Contiene la clase `MaquinaInspeccion`, la máquina del puesto de inspección. Posee el siguiente atributo:

- `self.peso_maximo` Peso máximo que acepta la máquina.

■ **Entregar** **Modificar** `consultas.py`

Contiene las dos consultas sobre la bitácora. No tiene clases, solo funciones.

■ **Modificar** `main.py`: ejecuta las cuatro partes de la actividad. Puedes modificarlo mientras desarrollas, pero **no lo entregas**: se corrige con la versión original.

■ **No modificar** `salida_esperada.txt` y `datos/cajas.csv`.

- **No modificar** `clases/fabrica.py`

Contiene las clases `Caja`, `Registro` y `CajaRechazadaError`.

La clase `Caja` representa un paquete que entra a la cinta, con los datos tal como vienen impresos en su etiqueta. Los tres atributos son `str`, porque es la máquina la que decide si sirven:

- `self.codigo` Código impreso en la caja.
- `self.nombre` Nombre del producto.
- `self.peso` Peso declarado, como texto.

La clase `Registro` es una línea de la bitácora: lo que quedó anotado después de pasar una caja por la máquina. Posee los siguientes atributos:

- `self.codigo` Código de la caja inspeccionada.
- `self.resultado` Aprobada o Rechazada.
- `self.detalle` Motivo del rechazo, o – si fue aprobada.

La excepción `CajaRechazadaError` es la que levanta la máquina. Su inicializador recibe `motivo`, y produce mensajes con el formato:

Caja rechazada: <motivo>

- **No modificar** `utils.py`: Contiene `codigo_valido`, que revisa el formato del código, y `cargar_cajas`, que lee el archivo de datos y entrega las cajas en el orden en que aparecen.

Parte 1. La cinta

Debes completar dos métodos en `clases/lista_ligada.py`: `agregar` y `retirar_primera`.

La cinta es una `ListaLigada`: una cadena de `Nodo` donde cada uno apunta al siguiente. Las cajas entran por la cola y salen por la cabeza.

La misma clase se usa dos veces: para la cinta y para la bitácora de registros. Por eso guarda objetos genéricos y no sabe qué hay dentro de sus nodos.

Una lista recién creada tiene `cabeza` y `cola` en `None`, y `largo` en cero.

- `def agregar(self, valor: Caja | Registro) -> None:`

Agrega `valor` al final de la lista dentro de un nuevo `Nodo`, dejando consistentes `cabeza`, `cola` y `largo`.

- `def retirar_primera(self) -> Caja | Registro:`

Elimina el primer nodo de la lista y retorna el **valor** que almacenaba. Debe dejar consistentes `cabeza`, `cola` y `largo`, incluso cuando la lista queda vacía. Si no hay ningún nodo, levanta `IndexError` con un mensaje que lo indique.

El método `__len__` ya viene implementado: entrega el valor que tú mantienes en `largo`, así que `agregar` y `retirar_primera` deben actualizarlo.

No puedes usar una lista de Python como estructura intermedia ni auxiliar.

Parte 2. Iterable e iterador

Una `ListaLigada` no se recorre con índices: para pasar de un elemento al siguiente hay que seguir los enlaces entre nodos. De eso se encarga `IteradorListaLigada`, que recuerda en qué nodo va el recorrido y avanza uno a uno desde la cabeza hasta la cola.

En `ListaLigada`:

- `def __iter__(self) -> IteradorListaLigada:`

Retorna un `IteradorListaLigada` posicionado al inicio de la lista. Cada llamada debe producir un iterador independiente, de modo que dos recorridos distintos no interfieran entre sí.

Y en `IteradorListaLigada`:

- `def __init__(self, cabeza: Nodo | None) -> None:`

Recibe el nodo donde comienza el recorrido.

- `def __iter__(self) -> IteradorListaLigada:`

Retorna la instancia misma del iterador (`self`).

- `def __next__(self) -> Caja | Registro:`

Entrega el valor guardado en el nodo en que está el recorrido y deja el iterador avanzado al siguiente. Cuando ya no quedan nodos, levanta `StopIteration`.

Parte 3. La máquina

La máquina revisa la etiqueta y el peso de cada caja. Si algo no cumple, levanta `CajaRechazadaError` con el motivo del rechazo. Debes modificar el archivo `clases/maquina.py`.

- En `MaquinaInspeccion`: `def inspeccionar(self, caja: Caja) -> None:`

Valida el código, el nombre y el peso de la caja, en el orden de la Tabla 1. En cuanto una validación falle, levanta `CajaRechazadaError` con el motivo correspondiente.

Para revisar el peso debes convertirlo a `float`. La conversión puede fallar, por lo que debes capturar `ValueError`. El máximo permitido está en `self.peso_maximo`.

Los motivos deben ser exactamente estos, y se aplican en este orden:

Condición	Motivo
El código no tiene el formato CJA-XXXX	código inválido
El nombre está vacío	nombre vacío
El peso no se puede convertir a <code>float</code>	peso no numérico
El peso es menor o igual que cero	peso menor o igual a cero
El peso supera <code>peso_maximo</code>	peso sobre el máximo

Tabla 1: Motivos de rechazo, en orden de aplicación.

La función `codigo_valido` ya está implementada en `utils.py`. Úsala para la primera validación.

No debes preocuparte por valores especiales de `float` como `nan` o `inf`: el archivo de datos no los contiene.

Parte 4. Consultas

Debes modificar el archivo `consultas.py`. Para ello, debes implementar las siguientes consultas usando **exclusivamente expresiones generadoras**:

- `registros_por_resultado(bitacora, resultado)`

Retorna un generador que entrega, en orden, los objetos `Registro` de la bitácora cuyo resultado coincide con el indicado. Por ejemplo, `registros_por_resultado(bitacora, "Aprobada")` entrega los registros de las cajas que pasaron la inspección.

- `generar_reporte(bitacora, resultado)`

Retorna un generador de `str` que, a partir de `registros_por_resultado`, describe cada registro con el formato:

`<codigo>: <detalle>`

Cómo probar tu avance

El archivo `main.py` ejecuta las cuatro partes de la actividad, en el orden en que conviene implementarlas. Si una parte todavía no está lista, Python mostrará el error en esa sección; puedes comentarla y continuar con las demás.

El archivo `salida_esperada.txt` contiene la salida que debe producir `main.py` con la implementación completa, por lo que puedes utilizarlo como referencia para verificar tus resultados. Ten presente que, si modificas `main.py`, la salida también cambiará.

Puntaje por sección

Los criterios indican los aspectos principales que serán evaluados y pueden subdividirse durante la corrección.

Criterio	Puntaje
agregar mantiene consistentes <code>cabeza</code> , <code>cola</code> y <code>largo</code> .	0,8 pts
<code>retirar_primera</code> desconecta la cabeza en todos los casos y maneja la cinta vacía.	1,0 pts
<code>ListaLigada.__iter__</code> crea un iterador independiente en cada llamada.	0,4 pts
<code>IteradorListaLigada</code> implementa el protocolo de iterador sin modificar la lista.	1,0 pts
<code>MaquinaInspeccion</code> valida en el orden indicado y levanta <code>CajaRechazadaError</code> con los motivos esperados.	1,0 pts
Las consultas emplean expresiones generadoras y cumplen con lo esperado.	0,8 pts
Descuentos: Se podrá aplicar un máximo de 1 punto de descuento por problemas de legibilidad, incumplimiento de PEP 8 o de las restricciones indicadas.	
Total	5,0 pts

Notas

- No puedes agregar librerías externas ni modificar los nombres de las clases, funciones, parámetros o atributos entregados.
- Tu código debe poder ejecutarse por consola mediante `python main.py` o `python3 main.py`.
- No debes manejar la ausencia del archivo de datos ni errores de lectura distintos de los descritos.
- Debes entregar únicamente los archivos marcados con **Entregar** en la sección de archivos, dentro de la carpeta `Actividades/AC2` de tu repositorio personal.
- La entrega debe encontrarse en la rama por defecto, `main`, de tu repositorio de Git.
- Compara la ejecución del programa con `salida_esperada.txt` antes de entregar.
- **Importante:** no se permite utilizar `except Exception` en esta actividad.

Objetivos de la actividad

- Levantar y capturar excepciones.
- Implementar y utilizar una lista ligada.
- Crear un iterable y iterador definiendo correctamente los métodos `__iter__` y `__next__`.
- Construir consultas mediante expresiones generadoras.